Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Semaine 3

Microservice Véhicules

Etudiant :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

Semaine :
3 / 7

25 mars 2026

Table des matières

1 Introduction et objectifs de la semaine **2**

2 Tests de l'API REST avec Postman **2**

2.1 CRUD Véhicules 2

2.2 Gestion des assignations 5

2.3 Bilan des tests 6

1. Introduction et objectifs de la semaine

Objectif de la semaine 3 : Développer le premier microservice fonctionnel du service véhicules en respectant les bonnes pratiques de qualité logicielle : tests, couverture de code, et observabilité intégrée dès le départ.

La semaine 3 du projet marque le passage de l'infrastructure au développement métier. L'équipe s'est concentrée sur la conception et l'implémentation du service de véhicules qui est le socle du système de gestion de flotte.

Les grandes étapes réalisées sont les suivantes :

1. Développement de l'**API REST** complète (CRUD véhicules avec validation des données)
2. Intégration d'**Apache Kafka** en tant que producer/consumer pour les événements métier
3. Création des **résolveurs GraphQL** pour l'agrégation future entre services
4. **Tests unitaires (>80% couverture) et d'intégration**
5. Instrumentation du service avec le **SDK OpenTelemetry** (traces, métriques, logs structurés)

2. Tests de l'API REST avec Postman

Objectif : Valider le bon fonctionnement de l'ensemble des endpoints du service Véhicules via des tests manuels réalisés avec **Postman**, couvrant les opérations CRUD ainsi que la gestion des assignations.

L'ensemble des routes exposées par le service Véhicules ont été testées manuellement à l'aide de Postman. Les tests couvrent les deux ressources principales : les **véhicules** et leurs **assignations**. Chaque requête a retourné le code HTTP attendu, confirmant la conformité de l'API aux standards REST.

2.1. CRUD Véhicules

POST /vehicles — Ajout d'un véhicule

La création d'un véhicule s'effectue via une requête `POST` sur `/vehicles`. Le corps de la requête contient les informations du véhicule (`plateNumber`, `brand`, `model`, `fuelType`, `mileageKm`, `vin`, `payloadCapacityKg`, `cargoVolumeM3`). Le service retourne un **201 Created** avec l'objet créé, incluant l'UUID généré automatiquement et le statut par défaut `available`.

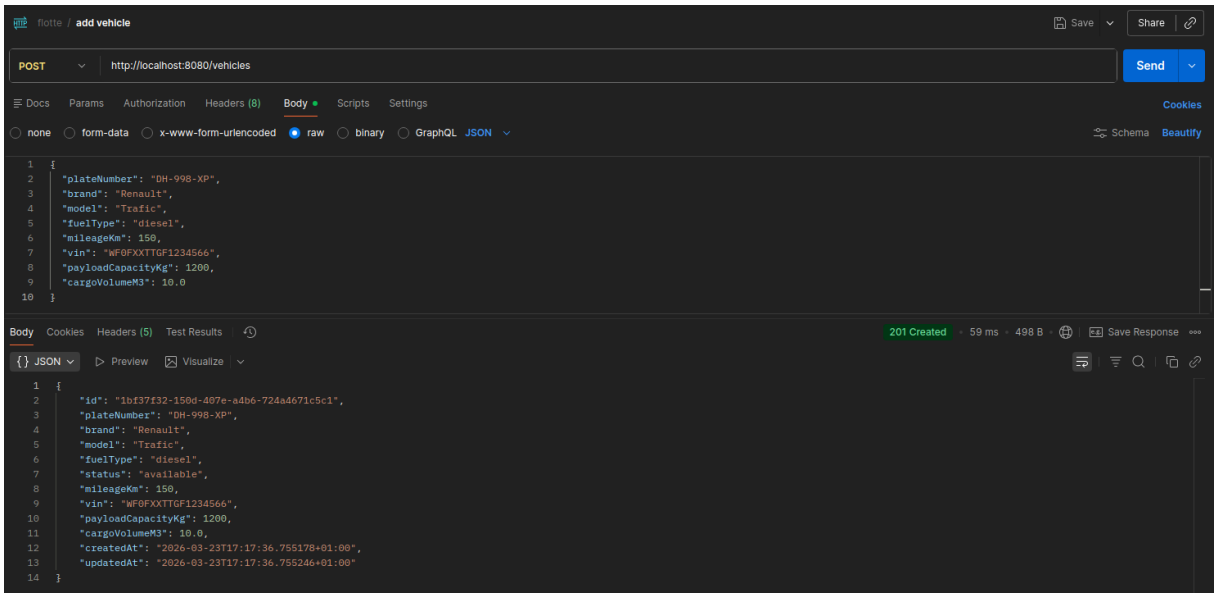

FIGURE 1 – POST /vehicles — Création d'un véhicule (201 Created)

	id [PK] uuid	plate_number character varying (20)	brand character varying (100)	model character varying (100)	fuel_type fuel_type	mileage_km integer	status vehicle_status
1	1bf37f32-150d-407e-a4b6-724a4671c5c1	DH-998-XP	Renault	Trafic	diesel	150	on_delivery
2	277223db-85d2-4543-8542-9c179ed393...	DH-999-XP	Ford	Transit	diesel	200	in_maintenance

FIGURE 2 – POST /vehicles — Détails

GET /vehicles — Liste de tous les véhicules

Une requête `GET` sur `/vehicles` retourne la liste des véhicules non supprimés. Le mécanisme de **soft delete** est ici mis en évidence : seuls les véhicules actifs sont retournés, même si la base de données en contient davantage. Le serveur répond avec un **200 OK**.

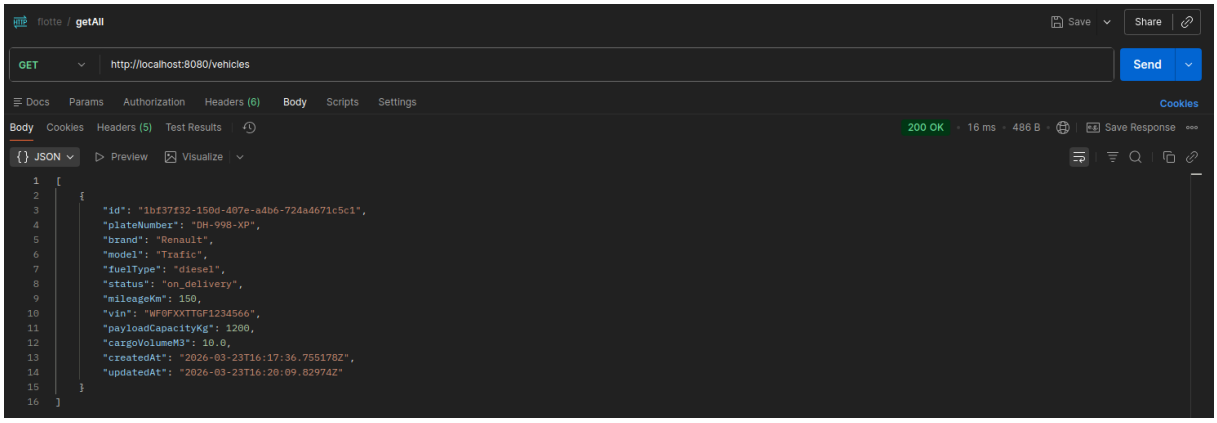

FIGURE 3 – GET /vehicles — Récupération de la liste des véhicules (200 OK)

GET /vehicles/ :id — Récupération par identifiant

La récupération d'un véhicule précis s'effectue en passant son UUID en paramètre de chemin. Le service retourne un **200 OK** avec l'intégralité des champs du véhicule correspondant.

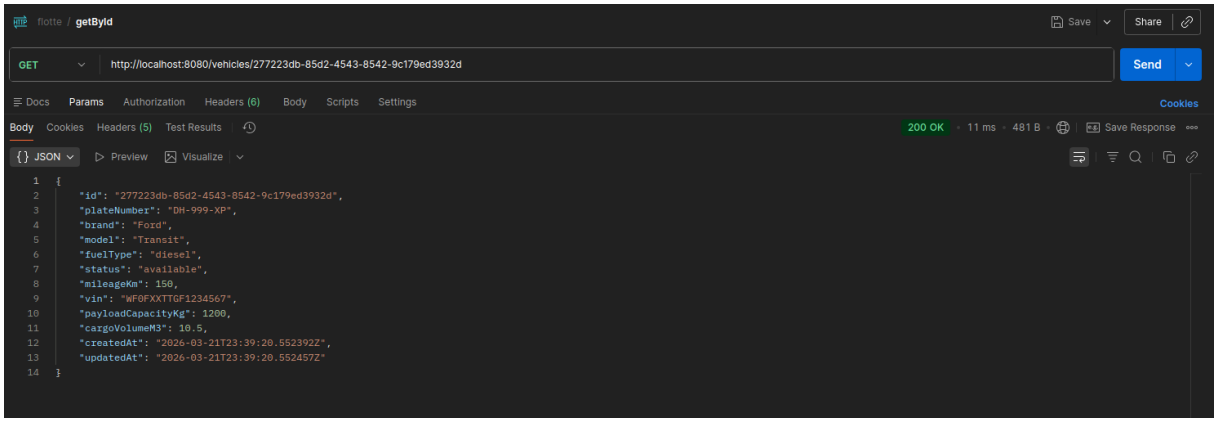

FIGURE 4 – GET /vehicles/ :id — Récupération d'un véhicule par UUID (200 OK)

PUT /vehicles/ :id — Mise à jour complète

La mise à jour d'un véhicule via `PUT` remplace les champs modifiables (`mileageKm`, `brand`, `model`, etc.). Le service retourne un **200 OK** avec les données mises à jour et un `updatedAt` rafraîchi.

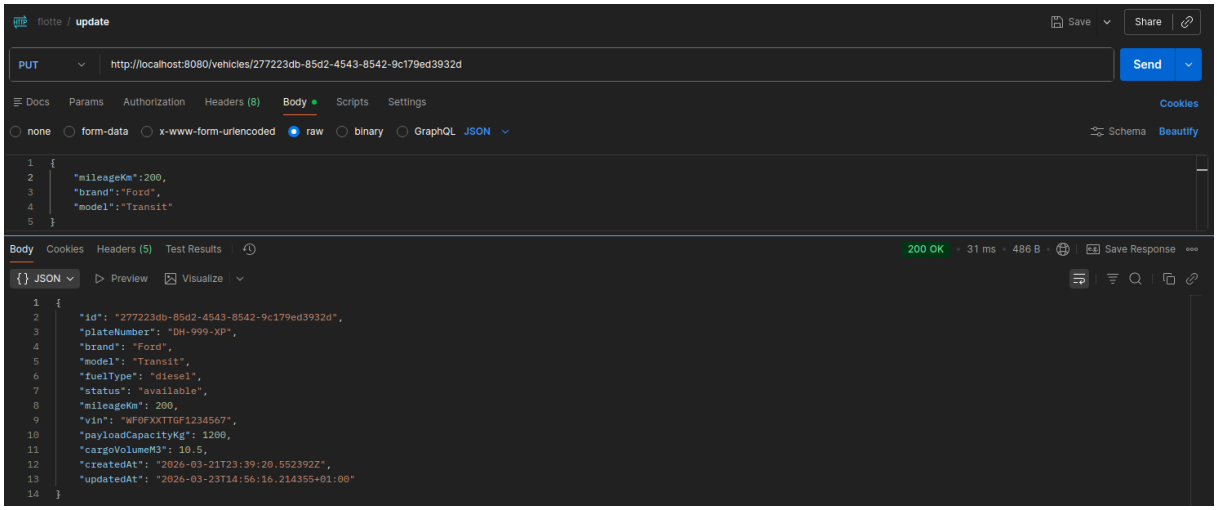

FIGURE 5 – PUT /vehicles/ :id — Mise à jour complète d'un véhicule (200 OK)

PATCH /vehicles/ :id/status — Mise à jour du statut

Une route dédiée permet de modifier uniquement le statut d'un véhicule (ex. `available`, `in_maintenance`, `on_delivery`). Cette approche évite d'envoyer l'intégralité du corps pour un changement ponctuel. Le service répond avec un **200 OK** et retourne le véhicule avec son nouveau statut.

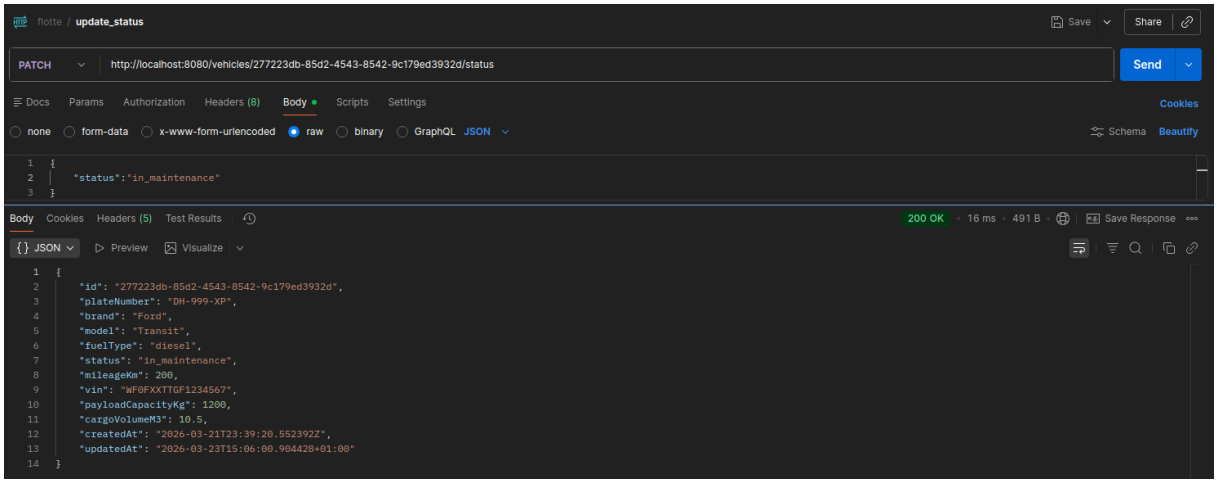

FIGURE 6 – PATCH /vehicles/ :id/status — Mise à jour du statut (200 OK)

DELETE /vehicles/ :id — Suppression logique

La suppression d'un véhicule est implémentée en **soft delete** : le véhicule n'est pas physiquement retiré de la base de données mais marqué comme supprimé. Le service retourne un **204 No Content**, et le véhicule disparaît des résultats des requêtes `GET`. La base de données (visible via pgAdmin) conserve bien les deux entrées, confirmant ce comportement.

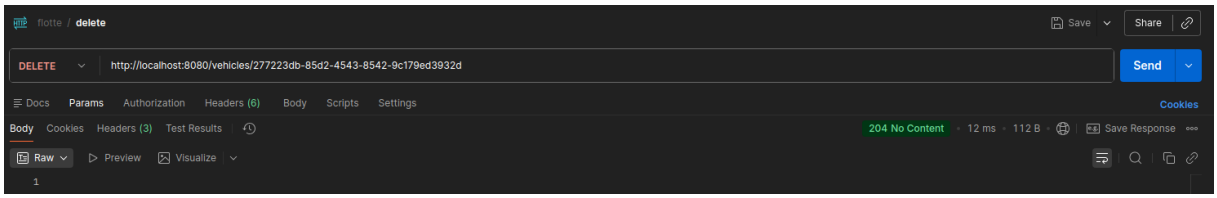

FIGURE 7 – DELETE /vehicles/ :id — Suppression logique (204 No Content)

2.2. Gestion des assignations

POST /vehicles/ :id/assignments — Ajout d'une assignation

L'assignation d'un conducteur à un véhicule s'effectue via une requête `POST` sur `/vehicles/:id/assi`
Le corps contient le `driverId`, des `notes` optionnelles et le `createdBy`. Le service répond avec un **201 Created** et retourne l'assignation créée avec sa date de début.

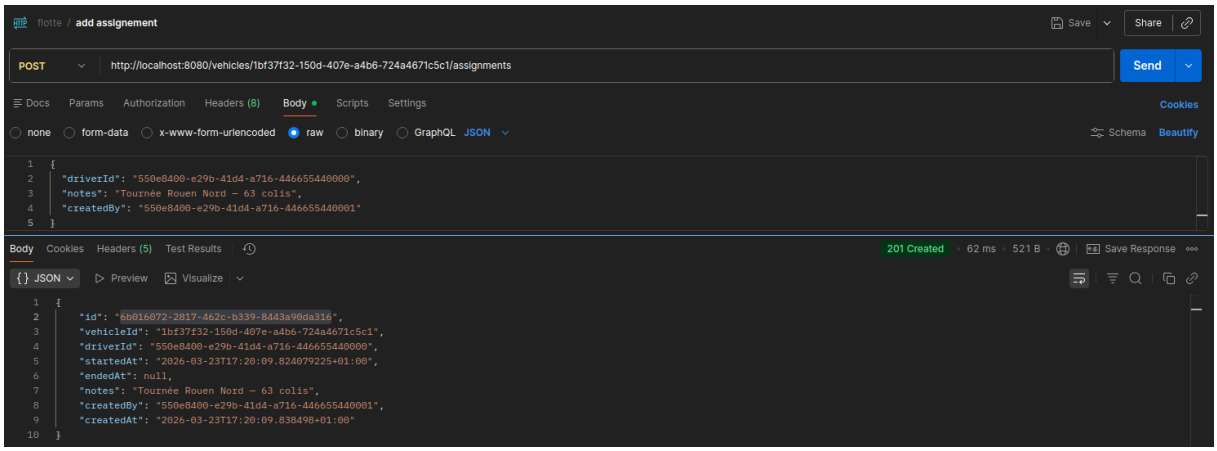

FIGURE 8 – POST /vehicles/ :id/assignments — Création d'une assignation (201 Created)

GET /vehicles/ :id/assignments — Liste des assignations

La récupération des assignations d'un véhicule retourne l'historique complet des affectations de conducteurs. Le champ `endedAt` à `null` indique une assignation en cours. Le service répond avec un **200 OK**.

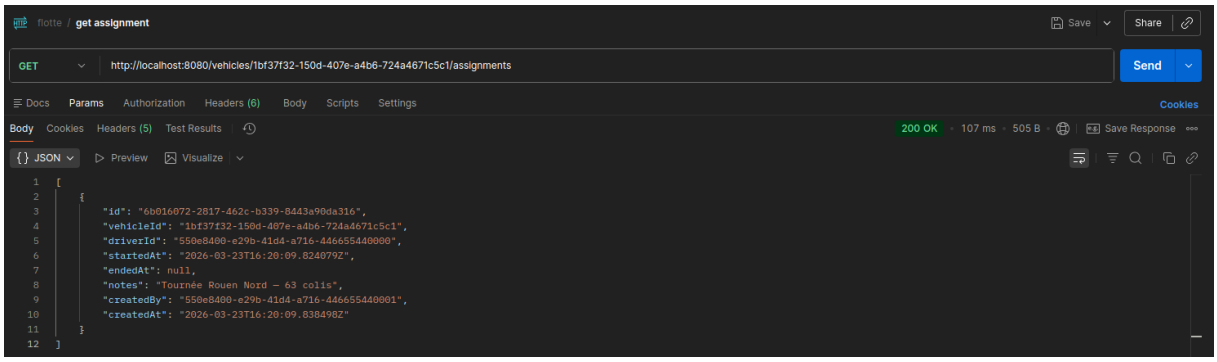

FIGURE 9 – GET /vehicles/ :id/assignments — Liste des assignations (200 OK)

2.3. Bilan des tests

Requête	Endpoint	Code attendu	Résultat
POST	/vehicles	201 Created	**OK**
GET	/vehicles	200 OK	**OK**
GET	/vehicles/ :id	200 OK	**OK**
PUT	/vehicles/ :id	200 OK	**OK**
PATCH	/vehicles/ :id/status	200 OK	**OK**
DELETE	/vehicles/ :id	204 No Content	**OK**
POST	/vehicles/ :id/assignments	201 Created	**OK**
GET	/vehicles/ :id/assignments	200 OK	**OK**

L'ensemble des 8 endpoints testés retournent les codes HTTP attendus. Le soft delete est correctement implémenté et le filtrage des véhicules supprimés fonctionne comme prévu.